TRANSFORMACIÓN DE DATOS CON PANDAS



Nombres:

Catalina Alarcon Zabala
Kizzye Vanessa Hidalgo Ariza
Andres Felipe Polo

DIFERENCIA ENTRE LA LIMPIEZA Y TRANSFORMACIÓN

LIMPIEZA

Se refiere a corregir o eliminar problemas que hacen que los datos sean inválidos o inconsistentes

Limpieza: eliminar nulos y duplicados

df = df.dropna(subset=["precio"])
df = df.drop_duplicates()

TRANSFORMACIÓN

Parte de datos ya limpios y los modifica para adaptarlos a un análisis específico

Transformación: derivar una nueva variable

*df["precio_con_iva"] = df["precio"] * 1.19*

SELECCIÓN DE COLUMNAS

En pandas puedes seleccionar una o varias columnas usando corchetes. Una sola columna devuelve una **Series**; una lista de columnas devuelve un **DataFrame**.

```
df["nombre"]                    # una columna (Series)
df[["nombre", "precio"]]        # varias columnas (DataFrame)
df.loc[:, "nombre":"precio"]    # rango de columnas por etiqueta
df.iloc[:, 0:3]                 # rango de columnas por posición
```

FILTROS SIMPLES

Un filtro simple aplica una condición booleana sobre una columna, y esa condición se usa para "**indexar**" el DataFrame completo.

df[df["precio"] > 50000]
df[df["categoria"] == "electrónica"]

FILTROS COMPUESTOS CON &, | Y ~

En lugar de usar **and**/**or**/**not** se usan los operadores bit a bit, y cada condición debe ir entre paréntesis:

```
df[(df["precio"] > 50000) & (df["categoria"] == "electrónica")]          # AND
 df[(df["categoria"] == "hogar") | (df["categoria"] == "oficina")]       # OR
df[~(df["categoria"] == "electrónica")]                                  # NOT
```

CREACIÓN DE COLUMNAS CALCULADAS

Se pueden crear columnas nuevas a partir de operaciones sobre columnas existentes, ya sea con operaciones aritméticas directas o con **apply()** para lógica más compleja.

```
df["ganancia"] = df["precio_venta"] - df["costo"]
df["margen_pct"] = (df["ganancia"] / df["precio_venta"]) * 100
df["nombre_mayus"] = df["nombre"].apply(lambda x: x.upper())
```

USO DE CONDICIONES CON NP.WHERE()

np.where(condición, valor_si_verdadero, valor_si_falso) permite crear columnas condicionales de forma vectorizada, más rápida que usar **apply()** con una función lambda para casos simples.

```
import numpy as np
df["estado"] = np.where(df["stock"] > 0, "Disponible", "Agotado")      # Anidado para varias condiciones

df["nivel"] = np.where(df["precio"] > 100000, "Alto",
                       np.where(df["precio"] > 50000, "Medio", "Bajo"))
```

ORDENAMIENTO DE REGISTROS

¿Qué es? Reorganizar las filas de una tabla según un criterio.

¿Para qué sirve? Encontrar máximos, mínimos, rankings y facilitar la lectura.

Pandas tiene dos métodos:

Método	Ordena según
sort_values()	Los valores de una o varias columnas
sort_index()	Las etiquetas del índice

Parámetros principales:

- by: columna o lista de columnas. Si es lista, la primera tiene prioridad y las demás desempatan.
- ascending: True de menor a mayor (defecto), False de mayor a menor.
- na_position: 'last' pone los vacíos al final (defecto), 'first' al inicio.
- inplace: False por defecto, es decir, no modifica la tabla original.

Ejemplo: ordenar las ventas de mayor a menor

```
df_ordenado = df.sort_values(by='ventas', ascending=False)
```

region	ciudad	anio	ventas
Norte	Bucaramanga	2025	700
Sur	Pasto	2025	600
Norte	Bucaramanga	2024	500
Norte	Cúcuta	2025	450
Sur	Neiva	2024	300

ORDENAMIENTO DE REGISTROS

Problema: algunas columnas tienen un orden lógico que no es alfabético.

```
df_nivel.sort_values('nivel')   # resultado: Alto, Bajo, Medio (incorrecto)
```

Solución: convertir la columna a tipo categórico y definir el orden.

```
df_nivel['nivel'] = pd.Categorical(df_nivel['nivel'],
                                   categories=['Bajo', 'Medio', 'Alto'],
                                   ordered=True)
df_nivel.sort_values('nivel')   # resultado: Bajo, Medio, Alto (correcto)
```

- categories: lista con el orden deseado.
- ordered=True: indica que ese orden tiene significado.

Ventajas adicionales de los datos categóricos:

- Ocupan menos memoria.
- Las búsquedas son más rápidas.

AGRUPAMIENTOS CON GROUP BY

¿Qué es? Una operación que agrupa filas según una o más columnas para calcular algo en cada grupo. Equivale al GROUP BY de SQL.

Funciona en tres pasos (dividir, aplicar, combinar):

1. Dividir: separa los datos en grupos según la clave.
2. Aplicar: ejecuta una función en cada grupo.
3. Combinar: junta los resultados en una nueva tabla.

Ejemplo: total de ventas por región

```
df.groupby('region')['ventas'].sum()
```

Paso a paso:

Paso	Norte	Sur
Dividir	500, 450, 700	300, 600
Aplicar (suma)	1650	900
Combinar	Norte: 1650	Sur: 900

Ojo: df.groupby('region') solo no muestra resultados; crea un objeto que espera una función.

AGREGACIONES SIMPLES Y MULTIPLES

¿Qué es una agregación? Una operación que convierte un conjunto de valores en un solo valor.

Agregaciones simples:

Función	Calcula
sum()	Suma
mean()	Promedio
median()	Mediana
min() / max()	Mínimo / Máximo
std()	Desviación estándar
count()	Cantidad de valores no vacíos
size()	Cantidad total de filas (incluye vacíos)
describe()	Resumen estadístico completo

Ojo: si hay columnas de texto, usar numeric_only=True o seleccionar solo la columna numérica.

Agregaciones múltiples con agg(): varias funciones a la vez.

```
df.groupby('region')['ventas'].agg(['sum', 'mean', 'count'])
```

region	sum	mean	count
Norte	1650	550	3
Sur	900	450	2

Agregación con nombre: permite nombrar cada resultado.

```
df.groupby('region').agg(total=('ventas', 'sum'),
                         promedio=('ventas', 'mean'))
```

region	total	promedio
Norte	1650	550
Sur	900	450

AGRUPAMIENTO POR VARIAS VARIABLES

¿Qué es? Agrupar usando dos o más columnas. Cada grupo es una combinación única de sus valores.

¿Para qué sirve? Cruzar información, por ejemplo ventas por región y por año.

```
df.groupby(['region', 'anio'])['ventas'].sum()
```

Resultado con índice jerárquico (MultiIndex):

region	anio	ventas
Norte	2024	500
	2025	1150
Sur	2024	300
	2025	600

Para convertir las claves en columnas normales:

```
df.groupby(['region', 'anio'], as_index=False)['ventas'].sum()
# o también
df.groupby(['region', 'anio'])['ventas'].sum().reset_index()
```

region	anio	ventas
Norte	2024	500
Norte	2025	1150
Sur	2024	300
Sur	2025	600

TABLAS PIVOTE CON PIVOT_TABLE

¿Qué es? Una tabla de doble entrada que resume datos, igual a una tabla dinámica de Excel.

Forma 1, a mano con groupby:

```
df.groupby(['region', 'anio'])['ventas'].sum().unstack()
```

Forma 2, con pivot_table:

```
pd.pivot_table(df, index='region', columns='anio',
               values='ventas', aggfunc='sum')
```

Las dos dan el mismo resultado:

region	2024	2025
Norte	500	1150
Sur	300	600

Parámetros:

- index: variable en las filas.
- columns: variable en las columnas.
- values: dato a resumir.
- aggfunc: operación (por defecto el promedio).
- fill_value: valor para reemplazar vacíos.
- margins=True: agrega fila y columna All con totales.

Con margins=True:

region	2024	2025	All
Norte	500	1150	1650
Sur	300	600	900
All	800	1750	2550

pivot vs pivot_table: pivot() solo reorganiza y da error con combinaciones repetidas; pivot_table() las agrega.

UNIÓN DE TABLAS CON MERGE

¿Qué es? Combinar dos tablas enlazando filas mediante una columna en común (clave). Equivale al JOIN de SQL.

Tablas de ejemplo:

Tabla 1: ventas

region	ventas
Norte	1650
Sur	900

Tabla 2: metas

region	meta
Norte	1500
Centro	1000

Clave: region. Norte está en ambas, Sur solo en ventas, Centro solo en metas.

pd.merge(ventas, metas, on='region', how='...')

El parámetro how define qué filas se conservan:

how='inner': solo las regiones que están en las dos tablas (es el valor por defecto)

region	ventas	meta
Norte	1650	1500

how='left': todas las regiones de la tabla izquierda (ventas)

region	ventas	meta
Norte	1650	1500
Sur	900	NaN

how='right': todas las regiones de la tabla derecha (metas)

region	ventas	meta
Norte	1650	1500
Centro	NaN	1000

how='outer': todas las regiones de ambas tablas

region	ventas	meta
Norte	1650	1500
Sur	900	NaN
Centro	NaN	1000

CONCATENACIÓN CON CONCAT

¿Qué es? Pegar dos o más tablas una debajo de la otra (o una al lado de la otra), sin buscar coincidencias por una clave.

¿En qué se diferencia de merge? merge empareja filas usando una columna en común; concat solo apila las tablas tal como vienen.

Ejemplo: tenemos las ventas de cada año en tablas separadas y queremos una sola tabla con todo.

Tabla 1: ventas_2024

region	año	ventas
Norte	2024	1650
Sur	2024	900

Tabla 2: ventas_2025

region	año	ventas
Norte	2025	1800
Sur	2025	950

Concatenación vertical (axis=0): pone una tabla debajo de la otra. Es la opción por defecto.

pd.concat([ventas_2024, ventas_2025])

Resultado:

índice	region	año	ventas
0	Norte	2024	1650
1	Sur	2024	900
0	Norte	2025	1800
1	Sur	2025	950

Ojo: el índice se repite (0, 1, 0, 1) porque cada tabla conserva el suyo. Esto puede causar errores al buscar filas con loc.

Solución: agregar ignore_index=True para que el índice se reinicie y quede 0, 1, 2, 3.

pd.concat([ventas_2024, ventas_2025], ignore_index=True)

Parámetros principales:

- objs: la lista de tablas a unir, siempre entre corchetes [ ].
- axis: 0 apila filas (vertical), 1 pega columnas (horizontal).
- ignore_index: True reinicia el índice.
- keys: agrega una etiqueta para saber de qué tabla viene cada fila.
- join: 'outer' conserva todas las columnas (defecto), 'inner' solo las comunes.

Concatenación horizontal (axis=1): pega las columnas de una tabla al lado de las de la otra, alineando las filas por el índice.

```
pd.concat([ventas_2025, metas_2025], axis=1)
```

Ojo: si una columna no existe en alguna de las tablas, esos espacios quedan vacíos (NaN). Por eso, antes de concatenar, conviene revisar que las columnas tengan los mismos nombres.

FORMATO ANCHO Y LARGO

¿Qué es? Son dos formas de organizar exactamente los mismos datos. Cambia la forma de la tabla, no la información.

Formato ancho

Cada año es una columna. Tiene pocas filas y es fácil de leer, como un reporte de Excel.

region	2024	2025
Norte	1650	1800
Sur	900	950

Formato largo

Una fila por cada región y año. Ideal para groupby y gráficos.

region	año	ventas
Norte	2024	1650
Sur	2024	900
Norte	2025	1800
Sur	2025	950

De ancho a largo con melt(): melt significa “derretir”. Las columnas de los años se “derriten” y se convierten en filas.

```
df_largo = df_ancho.melt(id_vars='region',
                         var_name='anio', value_name='ventas')
```

Parámetros:

- id_vars: columnas que se quedan fijas (identifican cada fila).
- value_vars: columnas que pasan a ser filas (por defecto, todas las demás).
- var_name: nombre de la columna con los antiguos encabezados.
- value_name: nombre de la columna con los valores.

region	2024	2025
Norte	1650	1800
Sur	900	950



region	año	ventas
Norte	2024	1650
Sur	2024	900
Norte	2025	1800
Sur	2025	950

De largo a ancho con pivot(): hace el proceso contrario. Toma los valores de una columna y los vuelve encabezados.

```
df_ancho = df_largo.pivot(index='region', columns='año',
                          values='ventas').reset_index()
```

- index: lo que queda en las filas.
- columns: columna cuyos valores pasan a ser encabezados.
- values: el dato que llena la tabla.

Ojo: si hay combinaciones repetidas (dos filas Norte-2024), pivot da error. Ahí se usa pivot_table con aggfunc, como vimos antes.

¿Cuándo usar cada formato?

	Formato ancho	**Formato largo**
Sirve para	Reportes y presentar resultados	graficos y bases de datos
Forma	Muchas columnas, pocas filas	Pocas columnas, muchas filas

DOCUMENTACIÓN DE TRANSFORMACIONES

¿Por qué documentar? Cada filtro, merge o cambio de formato modifica los datos. Si no queda registrado, nadie (ni uno mismo dentro de un mes) sabe cómo se llegó al resultado.

Objetivo: que el análisis sea reproducible, es decir, que otra persona pueda repetirlo y obtener lo mismo.

1. No tocar los datos originales
Trabajar sobre una copia: df_limpio = df.copy()

2. Comentar cada paso
Explicar por qué se hace, no solo qué se hace (con #).

3. Usar nombres claros
df_ventas_2025 en vez de df2 o prueba_final.

4. Verificar antes y después
Revisar df.shape para saber cuántas filas se perdieron.

Ejemplo de un paso a paso documentado:

```
# 1. Cargar los datos originales (no se modifican)
df = pd.read_csv('ventas.csv')
print(df.shape)   # (120, 4)
# 2. Quitar filas sin valor de ventas
df_limpio = df.dropna(subset=['ventas'])
print(df_limpio.shape)   # (112, 4)
# 3. Dejar solo las ventas del 2025
df_2025 = df_limpio[df_limpio['año'] == 2025]
print(df_2025.shape)   # (58, 4)
```

Registro de cambios:

Paso	Filas
Datos originales	120
Quitar vacíos	112
Solo año 2025	58

Tip: en Jupyter o Colab se pueden usar celdas de texto (Markdown) como títulos para explicar cada sección del análisis.

ALMACENAMIENTO DE TRANSFORMACIONES

¿Qué es? Guardar en un archivo las tablas ya transformadas, para no repetir todo el proceso cada vez y poder compartir los resultados.

Formato	Método	Ventaja	Desventaja
CSV	to_csv()	Universal, se abre en cualquier programa	No guarda los tipos (fechas, categorías)
Excel	to_excel()	Fácil de compartir con quien no programa	Más lento y pesado
Parquet	to_parquet()	Rápido, liviano y conserva los tipos	No se abre con doble clic
Pickle	to_pickle()	Guarda el DataFrame tal cual	Solo se puede leer con Python

Para leerlos de nuevo se usa la función equivalente: read_csv(), read_excel(), read_parquet() o read_pickle().

¡MUCHAS GRACIAS!